PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC3533 — Computación de Alto Rendimiento — 2026-1

Tarea 1: Paralelismo Implícito y *Task Parallelism*

Viernes 27 de Marzo de 2026

En grupos de tres personas, desarrollar y entregar el informe respectivo. Los experimentos deben realizarse en al menos dos computadores distintos y los resultados deben compararse entre sí.

Plazo de Entrega : Viernes 17 de abril de 2026, 23:59

Configuración del entorno Python. Los scripts provistos requieren Python 3.12 con las librerías `numpy`, `scikit-learn`, `threadpoolctl` y `matplotlib`. Se recomienda usar **Miniconda**:

1. Descargar e instalar Miniconda desde `https://docs.conda.io/en/latest/miniconda.html`.
2. Crear un entorno con Python 3.12 e instalar las librerías necesarias:

```
conda create -n tarea1-hpc python=3.12 -y
conda activate tarea1-hpc
conda install numpy scikit-learn matplotlib threadpoolctl -y
```

3. Verificar la instalación ejecutando el script serial de referencia:

```
python3 src/kmeans.py
```

Los experimentos de las preguntas siguientes deben repetirse para $N \in \{10\,000,\ 100\,000,\ 500\,000\}$ muestras sintéticas, comparando los resultados obtenidos en cada caso.

1. **[90 pts]** *K-means con `multiprocessing`*

El algoritmo k-means divide un conjunto de N puntos $\{x_i\}_{i=1}^{N} \subset \mathbb{R}^d$ en k *clusters* $\{C_j\}_{j=1}^{k}$, minimizando la inercia total:

$$J = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2, \qquad \mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x$$

El algoritmo alterna entre dos pasos hasta convergencia: (1) asignar cada punto al centroide μ_j más cercano, y (2) recalcular los centroides. En la práctica se ejecuta `n_init` veces con distintas inicializaciones aleatorias, retornando la solución de menor inercia.

Dado que cada valor de k se optimiza de forma independiente, el problema es naturalmente paralelizable como *task parallelism*: cada tarea corresponde a un ajuste de k-means para un valor de k distinto. Se usa `sklearn.cluster.KMeans` con datos sintéticos generados por `make_blobs`. En el directorio `src/` se proveen los scripts `kmeans.py` (versión serial de referencia), `kmeans_demo0.py` y `kmeans_demo1.py`.

(a) **[5 pts]** Declare el número de cores físicos y lógicos (threads) de su computador, e indique la velocidad de reloj del procesador.

(b) **[10 pts]** Explique cómo `multiprocessing.Pool` crea los procesos workers. ¿Cómo se asigna memoria al crear un worker? ¿Qué ocurre con el arreglo `X` al lanzar p workers?[1]

(c) **[10 pts]** Al ejecutar `kmeans_demo0.py` se imprime en pantalla la información del *backend* de threading que utiliza `scikit-learn` internamente, mediante `threadpool_info()`. Observe el *output*, identifique el *backend* reportado e investigue qué es: ¿qué librería implementa ese *backend*? ¿Qué función cumple dentro de `scikit-learn` y `numpy`? ¿Por qué es relevante conocerlo al paralelizar con `multiprocessing`?

(d) **[15 pts]** Ejecute `kmeans.py` para obtener el tiempo serial de referencia T_s. Luego, ejecute los scripts `kmeans_demo0.py` y `kmeans_demo1.py` con $p \in \{1, \ldots, 8\}$ workers y registre los tiempos. ¿Para qué sirve `threadpool_limits`? Basándose en sus resultados, explique qué ocurre cuando no se usa y defina el concepto de *oversubscription*.

[1] La respuesta puede variar dependiendo del sistema operativo.

(e) **[20 pts]** Usando los resultados de `kmeans_demo1.py`, calcule para cada p:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}$$

Grafique $T(p)$, $S(p)$ y $E(p)$ vs p comparando con la curva ideal. Comente los resultados.

(f) **[10 pts]** Calcule y grafique el *overhead* $T_o(p)$ en función de p. Identifique y comente las principales fuentes de *overhead* en este esquema.

(g) **[10 pts]** Ejecute experimentos variando simultáneamente el número de workers p y el número de threads internos t (`threadpool_limits`), con $p \cdot t \leq p_{\text{máx}}$ donde $p_{\text{máx}}$ es el número de cores lógicos. Grafique los tiempos de ejecución para cada combinación (p, t) y concluya cuál es la mejor combinación.

(h) **[10 pts]** Compare los resultados obtenidos en los dos computadores utilizados. ¿Qué diferencias, si es que hubo, fueron las más notables? Justifique a qué se deben.

Instrucciones de entrega:

- Entregar un documento PDF con sus resultados a través de Canvas.
- El uso de IA está permitido, mientras esté debidamente declarado en el informe.
- Se evaluará la claridad y legibilidad de los gráficos. Es importante identificar títulos, *labels*, leyendas y tamaño de fuente adecuados.